# ArtBench-10 Generative Modeling: VAE, DCGAN, and Diffusion

Nuno Batista and Miguel Cabral Pinto

University of Coimbra

**Abstract.** We design, train, and evaluate multiple generative model families on ArtBench-10: a VAE, a DCGAN (and cGAN variant), and diffusion-based models, including fine-tuning of a pretrained Google DDPM. Models are selected via Bayesian hyperparameter sweeps on a 20% subset and evaluated with FID/KID. Final results are reported for full-dataset training across 10 random seeds.

## 1 Introduction

We study generative modeling on ArtBench-10, a 10-class dataset of $32 \times 32$ paintings [1]. We implement and compare a VAE [2], a DCGAN and conditional variant (cGAN) [3], and diffusion-based models [4], including fine-tuning a pretrained Google DDPM. We evaluate sample quality with FID and KID and report results across 10 random seeds.

## 2 Problem Description

The target domain is ArtBench-10 [1], a curated benchmark of 50,000 training and 10,000 test images across 10 artistic styles. Each image is a $32 \times 32$ RGB painting.

The objective is to learn a generative model $p_\theta(x)$ that can sample plausible paintings while maintaining fidelity and diversity. Key challenges include:

- **Fidelity:** Avoiding overly smooth or noisy samples.
- **Diversity:** Covering multiple styles without collapse.
- **Stability:** Handling adversarial (GAN) and iterative (diffusion) training dynamics.

## 3 Methodology and Approach

### 3.1 Data Pipeline

The data pipeline handles the ArtBench-10 dataset ($32 \times 32$ RGB images, 10 styles). Images are converted from integer pixels to floating point and normalized to the range $[-1, 1]$ via

$$x' = 2 \cdot \frac{x}{255} - 1.$$

This normalization centers the data around zero and standardizes the dynamic range across models, which improves numerical conditioning for gradient-based optimization and aligns the data scale with common generative-model output parameterizations (e.g., a `tanh` output layer naturally produces values in $[-1, 1]$). For diffusion models, using a symmetric, zero-centered range is also convenient because the forward noising process is defined around zero-mean Gaussian noise.

During the experimentation phase, a 20% subset (10,000 images) was used for hyperparameter sweeps via Bayesian optimization. Once optimal parameters were identified, the final models were retrained on the full 50,000-image training set.

### 3.2 Model Families

The architectural design choices for each model family were tailored to the $32 \times 32$ resolution. Table 1 provides a comparative summary of the structural components across all implemented networks.

**Table 1.** Comprehensive architectural summary of the implemented generative models.

| Feature | VAE | DCGAN / cGAN | Diffusion | Google DDPM |
|---|---|---|---|---|
| **Backbone** | 4-Layer CNN | 4-Layer CNN | 2-Stage UNet | 4-Stage UNet |
| **Upsampling** | Transpose Conv | Transpose Conv | Transpose Conv | Upsample + Conv |
| **Downsampling** | Strided Conv | Strided Conv | Strided Conv | Strided Conv |
| **Normalization** | Batch Norm | Batch Norm | Group Norm | Group Norm |
| **Activation** | LeakyReLU (0.2) | ReLU (G) / LReLU (D) | SiLU (Swish) | SiLU (Swish) |
| **Bottleneck** | $2 \times 2 \times 512$ | $1 \times 1 \times 64$ | $8 \times 8 \times 512$ | $4 \times 4 \times 512$ |
| **Conditioning** | N/A | Class Embedding | Time + Class | Timestep |
| **Attention** | None | None | Bottleneck (Opt) | Multi-head |

**VAE** Our VAE implementation features a symmetric 4-layer convolutional architecture (32→16→8→4→2). Each layer employs Batch Normalization and LeakyReLU activations. We employ the reparameterization trick to allow for end-to-end backpropagation through the 256-dimensional stochastic bottleneck.

**DCGAN and cGAN** The GAN architectures follow DCGAN [3]. The cGAN variant incorporates a style embedding (dimension 64) concatenated with the noise vector in the generator and injected as an additional channel in the discriminator.

**Diffusion (From Scratch)** Our "TinyUNet" consists of two downsampling and two upsampling stages using Residual Blocks with Group Normalization and SiLU activations. Skip connections preserve high-frequency stylistic details.

**Diffusion (Pretrained Google DDPM)** We fine-tuned the `google/ddpm-cifar10-32` model [5], which uses a larger UNet with self-attention layers.

### 3.3 Implementation and Reproducibility

Our pipeline is configuration-driven via YAML files. We utilize Weights & Biases (W&B) for real-time metric logging. All experiments were conducted using PyTorch, with random seeds fixed to 67 for hyperparameter comparisons.

### 3.4 Theoretical Analysis of Hyperparameters

We summarize the swept hyperparameters and their primary effects on optimization stability, capacity, and sample quality.

#### Variational Autoencoders (VAE)

- **Learning rate (`lr`):** Controls optimization step size; too large can destabilize training and degrade reconstructions, while too small slows convergence.
- **Latent dimension (`latent_dim`):** Sets bottleneck capacity; larger values increase expressiveness but can weaken regularization and encourage blurry reconstructions if the decoder dominates.
- **Base channels (`base_channels`):** Scales network width and capacity; increasing it typically improves fidelity at higher compute cost.
- **KL weight ($\beta$):** Scales the KL regularizer in the VAE objective, i.e., it weights the pressure for $q_\phi(z \mid x)$ to match the prior $p(z)$. With a reconstruction loss term (here, effectively an $L2$-style distortion term) and a KL term, increasing $\beta$ reduces the information capacity of the latent (it discourages encoding fine detail in $z$), typically improving sampling from the prior and latent smoothness but worsening reconstruction fidelity. Decreasing $\beta$ does the opposite: it behaves more like a deterministic autoencoder ($\beta \to 0$), often improving reconstructions but producing a latent distribution that deviates from the prior, which can harm unconditional sample quality when sampling $z \sim p(z)$.
- **Batch size (`batch_size`):** Affects gradient noise and effective learning rate; larger batches stabilize optimization but reduce stochastic regularization.

#### Generative Adversarial Networks (GAN)

- **Learning rate (`lr`):** One of the most sensitive GAN hyperparameters; higher values can cause divergence or oscillations in the discriminator–generator game.
- **Latent dimension (`latent_dim`):** Controls the dimensionality of the generator input noise; too small can limit diversity, while too large can make mapping harder without improving quality.

- **Base channels (`base_channels`):** Changes generator/discriminator capacity; imbalance can lead to either an overpowered discriminator (vanishing gradients) or an underpowered discriminator (poor signal).
- **Discriminator updates (`d_updates_per_g`):** Sets the ratio of discriminator steps per generator step; increasing it can stabilize training when the discriminator is weak, but can also over-train the discriminator and stall generator learning.
- **Spectral normalization (`use_spectral_norm`):** Constrains discriminator Lipschitzness, often improving stability, but can reduce discriminator flexibility and slow learning depending on the regime.
- **Observed regime:** The best setting used `d_updates_per_g`=1, `use_spectral_norm`=false, and `latent_dim`=64. Samples remained prone to high-frequency artifacts consistent with transpose-convolution upsampling.

### Conditional GAN (cGAN)

- **Generator conditioning (concat in latent space):** Given a noise vector $z$ (dimension $d$) and a class label $y$, the generator embeds the label with an embedding layer $e(y)$ (dimension 64) and concatenates them $[z; e(y)]$. The concatenated vector is reshaped to $(d + 64) \times 1 \times 1$ and fed through the standard DCGAN transposed-convolution stack. This makes the class signal available from the first layer onward.
- **Discriminator conditioning (label as an image channel):** The discriminator embeds $y$ into a vector of size $32 \cdot 32$, reshapes it into a $1 \times 32 \times 32$ map, and concatenates it with the RGB image as an extra input channel (input becomes $4 \times 32 \times 32$). The discriminator therefore learns to score joint pairs $(x, y)$ rather than marginal realism of $x$.
- **Effect on optimization:** Conditioning increases discriminator capacity (it can use the label map as a shortcut feature). If the discriminator learns class-specific cues faster than the generator can match them, gradients to the generator become less informative, reducing both diversity and conditional fidelity.

### Diffusion Models

- **Learning rate (`lr`):** Controls how quickly the denoiser fits the noise-prediction objective; too large can harm stability or lead to overshooting, while too small slows convergence.
- **Base channels (`base_channels`):** Sets UNet width; larger models typically improve sample fidelity and style capture at increased compute.
- **Attention (`use_attention`):** When enabled, the TinyUNet inserts a multi-head self-attention block at the bottleneck feature map (after the mid residual block, at the lowest spatial resolution). Concretely, it computes $Q, K, V$ via a $1 \times 1$ convolution, flattens the $H \times W$ spatial grid into tokens, and applies scaled dot-product attention so each location can aggregate information from all other locations. The attention output is projected back with a $1 \times 1$

convolution and added residually. This can improve global coherence (e.g., consistent structure across distant regions) at relatively low cost because the bottleneck has the smallest $H \times W$.
- **Training length:** For the from-scratch model, extending training to 50 epochs produced the largest quality gains.
- **Fine-tuning:** For Google DDPM, a higher learning rate ($5 \times 10^{-4}$) improved adaptation from CIFAR-10 priors to ArtBench.

## 4 Experimental Setup

### 4.1 Hyperparameter Search

Hyperparameter selection was performed on a **20% training subset** (10,000 images) using **Bayesian optimization** (Weights & Biases sweeps). Each sweep run trained a candidate configuration and evaluated sample quality using **FID** (minimize). For computationally expensive diffusion-based models, sweep runs used reduced evaluation settings (e.g., fewer generated samples and shorter training) and the best configuration was then retrained with the final evaluation settings.

We used model-specific search spaces and budgets, following the rationale in our experiment plan:

- **VAE sweep:** {`lr`, `latent_dim`, `base_channels`, $\beta$, `batch_size`}, with an effective budget of roughly **50 runs** (out of 360 possible combinations), stopping when the best FID plateaued for approximately 10 consecutive runs.
- **DCGAN/cGAN sweep:** {`lr`, `latent_dim`, `base_channels`, `d_updates_per_g`, `use_spectral_norm`}, with a budget of roughly **30 runs** (out of 108 possible combinations), stopping on diminishing returns in best FID.
- **Custom Diffusion (TinyUNet) sweep:** restricted to the most impactful knobs {`lr`, `base_channels`, `use_attention`} (12 combinations), with about **20 runs** and reduced sweep settings (e.g., `eval_num_samples`=1000 and 20 epochs).
- **Google DDPM fine-tuning sweep:** {`lr`, `epochs`} (15 combinations), with about **20 runs** to balance adaptation vs. catastrophic forgetting.

To visualize the explored hyperparameter configurations and their performance trends, we used Weights & Biases sweep diagnostics. Figure 1 is reserved for **parallel coordinates** plots (one per model family). Additionally, we performed **fANOVA-based importance analysis** to estimate which parameters most strongly influenced FID; Figure 2 is reserved for the importance summaries (one per model family).

After each sweep, we selected the configuration with the lowest FID on the 20% subset. We then performed a **final evaluation** by training that best configuration across **10 random seeds** and computing FID/KID using **5,000 generated** samples versus **5,000 real** samples (KID computed over 50 random subsets of size 100). Finally, we retrained the best-performing model family on the **full 50k** training set using its best hyperparameters and repeated the 10-seed evaluation.

(a) Placeholder: W&B parallel coordinates (VAE)

(b) Placeholder: W&B parallel coordinates (DCGAN/cGAN)

(c) Placeholder: W&B parallel coordinates (TinyUNet Diffusion)

(d) Placeholder: W&B parallel coordinates (Google DDPM)

**Fig. 1.** Weights & Biases parallel coordinates plots for the Bayesian hyperparameter sweeps. Each panel summarizes the relationship between hyperparameter values and FID across runs for one model family. (Placeholders to be replaced with exported W&B figures.)

### 4.2 Training Protocol

Table 2 summarizes the final hyperparameters used for the 100% dataset training.

**Table 2.** Final hyperparameter configurations for full-dataset training.

| **Parameter** | **VAE** | **DCGAN** | **Diffusion** | **Google DDPM** |
|---|---|---|---|---|
| Learning Rate | $2 \times 10^{-3}$ | $2 \times 10^{-4}$ | $2 \times 10^{-4}$ | $5 \times 10^{-4}$ |
| Batch Size | 256 | 128 | 32 | 32 |
| Epochs | 20 | 20 | 50 | 10 |
| Latent Dim | 256 | 64 | 128 | 128 |
| Base Channels | 128 | 128 | 128 | 128 |
| Optimizer | Adam | Adam | AdamW | AdamW |

### 4.3 Evaluation Protocol

Following the project requirements, we generated 5,000 samples for each model and compared them against 5,000 real images.

| | |
|---|---|
| (a) Placeholder: fANOVA importance (VAE) | (b) Placeholder: fANOVA importance (DCGAN/cGAN) |
| (c) Placeholder: fANOVA importance (TinyUNet Diffusion) | (d) Placeholder: fANOVA importance (Google DDPM) |

**Fig. 2.** fANOVA parameter importance for the hyperparameter sweeps, reporting the estimated contribution of each hyperparameter to FID variance for each model family. (Placeholders to be replaced with exported analysis figures.)

- **FID:** Computed using the full sets of 5,000 generated and 5,000 real images [6].
- **KID:** Computed using 50 random subsets of size 100.
- **Statistical Robustness:** The entire protocol was repeated across **10 different random seeds**. We report Mean ± Standard Deviation.

## 5 Results and Analysis

### 5.1 Qualitative Results

Visual inspection of 64-sample grids reveals distinct model characteristics across the different families.

### 5.2 Quantitative Results

Table 3 presents the results of the 10-seed evaluation on the full dataset.

### 5.3 Training Dynamics

Loss curves (Figure 5) highlight different optimization behavior across model families:

- **GAN:** Adversarial losses oscillate and do not necessarily decrease monotonically, consistent with a non-stationary minimax objective.
- **Diffusion:** Noise-prediction MSE decreases smoothly during training, reflecting a supervised regression-style objective [4].

placeholder_vae.png

placeholder_dcgan.png

placeholder_cgan.png

placeholder_diffusion.png

**Fig. 3.** Qualitative samples (64-image grids) from the four main model families evaluated on ArtBench-10: (top-left) VAE, (top-right) DCGAN, (bottom-left) cGAN, and (bottom-right) custom TinyUNet diffusion. (Placeholders to be replaced with exported sample grids.)

### 5.4 Ablations and Discussion

Our results highlight several key trade-offs:

- **Unconditional vs. Conditional:** DCGAN significantly outperformed cGAN, suggesting that style-conditioning adds complexity that requires higher capacity or more training.
- **Diffusion Dominance:** Diffusion models achieved the best FID, benefiting from a stable MSE loss landscape and skip connections.
- **Perceptual Quality:** While TinyUNet Diffusion achieved the lowest FID (43.03), the **fine-tuned Google DDPM** produced the most coherent samples by visual inspection despite a higher FID (97.49).

placeholder_google_ddpm.png

**Fig. 4.** Samples from the fine-tuned Google DDPM, demonstrating superior structural coherence and recognizable shapes compared to the other models.

**Table 3.** Quantitative results (Mean ± Std Dev) across 10 random seeds.

| **Model** | **FID** ↓ | **KID** ($\times 10^{-3}$) ↓ |
|---|---|---|
| VAE | $355.26 \pm 1.32$ | $370.3 \pm 4.0$ |
| DCGAN | $91.02 \pm 0.65$ | $69.0 \pm 1.5$ |
| cGAN | $393.68 \pm 1.36$ | $329.1 \pm 2.6$ |
| Diffusion (TinyUNet) | $\mathbf{43.03 \pm 0.51}$ | $\mathbf{26.7 \pm 1.0}$ |
| Google DDPM (Fine-tuned) | $97.49 \pm 1.26$ | $90.5 \pm 1.5$ |

- **GAN Artifacts:** Despite good FID, DCGAN samples suffer from checkerboard noise, highlighting that metrics do not always align with human perception of quality.
- **VAE smoothness:** VAEs optimized with an $L2$-style reconstruction loss are mean-seeking: when multiple high-frequency outcomes are plausible, the decoder is rewarded for predicting an average, which tends to wash out texture and contrast. This effect belongs to the model–loss combination (not a single sweep knob), and helps explain the qualitatively smoother samples.

*Why Google DDPM can look better but score worse.* FID and KID evaluate distributional similarity in a fixed feature space rather than directly measuring perceived coherence [6]. The fine-tuned Google DDPM starts from a strong pretrained prior (CIFAR-10), which can preserve globally coherent structure after adaptation; however, its samples can remain partially misaligned with ArtBench feature statistics (e.g., texture and style cues), leading to higher FID/KID de-

placeholder_loss_curves.png

**Fig. 5.** Comparison of training signatures: The DCGAN (top) shows adversarial fluctuations, while the Diffusion model (bottom) shows smooth convergence on the stationary noise-MSE objective.

spite visually plausible outputs. In addition, a model may produce a subset of high-quality samples while under-covering modes (reduced diversity), which is penalized by FID/KID even when qualitative grids appear strong.

## 6 Conclusion

On ArtBench-10, diffusion-based models provided the best overall performance under our evaluation protocol. Quantitatively, the custom TinyUNet diffusion model achieved the lowest FID, while qualitatively the fine-tuned Google DDPM produced the most coherent samples.

## Use of External Resources and AI Tools

This project utilized ArtBench-10 (Kaggle). We used the `diffusers` library and `google/ddpm-cifar10-32` (Hugging Face). Gemini assisted in LaTeX formatting and analysis verification.

## References

1. Liao, P. et al. (2022). The ArtBench Dataset. arXiv:2206.11404.

2. Kingma, D. P. et al. (2013). Auto-Encoding Variational Bayes. arXiv:1312.6114.
3. Radford, A. et al. (2015). Unsupervised Representation Learning with DCGANs. ICLR 2016.
4. Ho, J. et al. (2020). Denoising Diffusion Probabilistic Models. NeurIPS 2020.
5. Google. (2020). DDPM CIFAR-10 32x32. Hugging Face.
6. Heusel, M. et al. (2017). GANs Trained by a Two Time-Scale Update Rule. NIPS 2017.